Tabular Machine-Learning Vetting of NASA Kepler Objects of Interest: A Reproducible Seven-Classifier Comparison with Leakage-Free Cross-Validation, Threshold Tuning and Three-Way Feature-Selector Consensus

Andersson David Sánchez Méndez
Master in Data Science
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia
andersson.sanchez-m@mail.escuelaing.edu.co

Cristian Santiago Pedraza Rodríguez
Master in Data Science
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia
cristian.pedraza-r@mail.escuelaing.edu.co

Mario Julián Cañón Ayala
Course Coordinator, MLEA_M
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia

Héctor Javier Hortúa Orjuela
Lecturer, MLEA_M
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia

***Abstract*—We present a reproducible machine-learning pipeline that classifies Kepler Objects of Interest (KOIs) from the NASA Exoplanet Archive Cumulative Table as either genuine planet *candidates* or *false positives*, using only the photometric and stellar features available *before* the official vetting decision is made. Seven classifier families spanning seven distinct inductive biases — Logistic Regression, k-Nearest Neighbours, a single CART Decision Tree, a Radial-Basis-Function Support Vector Machine, Random Forest, XGBoost and a Multi-Layer Perceptron — are trained on a stratified 72/8/20 split of 9 564 KOIs after the explicit removal of ten label-leaking columns identified by audit of the Kepler vetting protocol. The minority/majority ratio is checked at run time and SMOTE is applied only when the empirical ratio falls below a configurable threshold; for the cumulative KOI table at $\rho \approx 0.97$ it remains disabled. The two most expressive non-linear models are tuned with 5-fold cross-validated `GridSearchCV` scoring on F_1. Tuned XGBoost wins every test-set metric ($F_1 = 0.860$, ROC-AUC $= 0.936$, MCC $= 0.721$), followed in order by Random Forest, RBF-SVM, the tuned MLP, the single Decision Tree, k-NN and Logistic Regression. Threshold tuning by Youden's J shifts the decision cut-off from 0.50 to 0.43 and lifts recall from 0.869 to 0.899 at a $\sim$2.3 pp precision cost. Three independent feature-selection families — filter (`SelectKBest` with ANOVA F), wrapper (Recursive Feature Elimination with a Logistic Regression base estimator) and embedded (`SelectFromModel` on a Random Forest) — converge on `koi_model_snr`, `koi_prad` and `koi_depth` as essential, corroborating the XGBoost gain ranking. A Gaussian Mixture Model selected by the Bayesian Information Criterion, together with t-SNE and UMAP projections, characterises the geometry of the test split, and a learning-curve analysis shows that the XGBoost training and cross-validated F_1 scores converge before the full training set is exhausted. The full code, executed Jupyter notebooks, saved pipeline, design system, 20 figures and 78 unit tests are released under Apache 2.0 and reproduce deterministically with `RANDOM_SEED = 42`. The work continues, with course-grade methodology, a Global Finalist entry of the NASA Space Apps Challenge 2025 by team *ECI Centauri*.**

***Index Terms*—Exoplanet detection, Kepler, Kepler Objects of Interest, binary classification, XGBoost, Random Forest, Support Vector Machine, Multi-Layer Perceptron, decision tree, gradient boosting, SMOTE, data leakage, GridSearchCV, Matthews Correlation Coefficient, threshold tuning, Youden's J statistic, feature selection, Gaussian Mixture Model, t-SNE, UMAP, reproducible machine learning.**

I. Introduction

The Kepler space telescope monitored the brightness of more than 150 000 stars over its primary mission and produced almost 10 000 *Kepler Objects of Interest* (KOIs): periodic dimming events that *may* correspond to a planet transiting in front of its host star, but that are equally consistent with eclipsing binaries, stellar variability or instrumental artefacts [1], [15]. Each KOI is curated in the NASA Exoplanet Archive Cumulative Table [2] together with photometric transit parameters, stellar properties of the host, and a final vetting disposition produced by a partly manual NASA pipeline.

Automating the triage step — deciding whether a KOI is worth promoting to a follow-up observation queue — has direct scientific impact. A reliable classifier reduces the human-vetting load and accelerates the subsequent confirmation campaigns [3], [5]. The objective of this paper is therefore to produce, evaluate and *reproduce* a strong tabular baseline for that task while explicitly avoiding the data-leakage trap that has inflated metrics in earlier studies.

The work also continues, with full course-grade methodology, the *Exoplanet Hunter AI* prototype that the ECI Centauri team submitted to the NASA Space Apps Challenge 2025 and that earned a Global Finalist position. That hackathon system shipped a working web application and a light-curve CNN in 48 hours; here we focus on the tabular vetting question and on the rigour required by the *MLEA_M — Machine Learning* course of the Master in Data Science at Escuela Colombiana de Ingeniería Julio Garavito (ECI).

Contributions: The paper makes six concrete contributions:

1) An explicit, audit-grade list of *ten* label-leaking columns that must be removed from the KOI cumulative table before training, grouped into the three leakage families (*label*, *target-defining flags*, *identifier*) we propose as a taxonomy applicable to any vetted-by-humans scientific table.
2) A head-to-head comparison of seven algorithm families spanning seven distinct inductive biases — linear (LR), local (k-NN), single-tree partitioning (CART), margin-based with the kernel trick (RBF-SVM), bagging (RF), boosting (XGBoost) and differentiable function approximation (MLP).
3) Hyper-parameter tuning by 5-fold cross-validated `GridSearchCV` with F_1 as the primary scorer on the two highest-capacity non-linear models, plus targeted smaller grids for the remaining five.
4) A three-way feature-selector consensus study — filter, wrapper and embedded — corroborating the XGBoost gain ranking and providing transferable engineering guidance on which features survive every selection family.
5) Threshold tuning via Youden's J statistic, motivated by the cost asymmetry between a missed planet and a wasted follow-up observation, with the chosen operating point anchored in the validation ROC curve.
6) A fully reproducible open-source release: seven reusable `src/` modules, 78 unit tests, six executed Jupyter notebooks, twenty matplotlib/seaborn figures, the serialised best pipeline as a `joblib` artefact, an IEEE LaTeX paper, and a parallel HTML/CSS design system with a 20-slide defense deck.

The remainder of the paper is organised as follows. Section II surveys related work on tabular and light-curve KOI classification, gradient-boosted trees on tabular data, and the class-imbalance literature. Section III describes the dataset and the leakage-removal protocol with the proposed taxonomy. Section IV formalises the preprocessing pipeline. Section VI introduces the seven classifiers and the hyper-parameter-tuning protocol. Section V gives the system architecture in one figure. Section VII states the evaluation protocol. Section VIII presents the leaderboard, the ROC/PR curves, the confusion-matrix grid, feature importance, and the three-way feature-selector consensus. Section IX analyses threshold tuning. Section X reports the unsupervised course-extension geometry analyses (GMM, t-SNE, UMAP) and the learning-curve diagnostic. Section XI discusses why XGBoost wins and why the MLP loses; Section XII addresses application and impact; Section XIII closes with limitations and future work.

TABLE I: Target distribution before any preprocessing.

Class	**Count**	**Fraction**
FALSE POSITIVE	4 847	50.7%
CANDIDATE	4 717	49.3%
Total	9 564	100.0%

II. Related Work

A. Tabular vs. light-curve classification

Two streams of work address KOI classification: *tabular* models that consume engineered features from the Kepler pipeline (this work, the `Robovetter` family [4]) and *light-curve* models that consume raw flux time-series with convolutional or transformer architectures [3], [5]. The two are complementary; tabular models are cheap and explainable, light-curve models capture morphology that the table cannot.

B. Tree-based methods on tabular data

Gradient-boosted decision trees have dominated tabular benchmarks for two decades [7], [8], while Random Forest [6] provides a strong bagging baseline. Recent foundation-model approaches such as TabPFN [9] reach or exceed gradient-boosted performance on small datasets ($\leq 10\,000$ samples) without explicit hyper-parameter search; we discuss them as future work in Section XIII but keep XGBoost as the headline tuned model because it is the de-facto industry choice and is what the course covered.

C. Class imbalance

The KOI cumulative table is moderately imbalanced ($\approx 49\%/51\%$). When the minority/majority ratio falls below a tunable threshold we apply SMOTE [11] on the training set only — a guard against the inflation of cross-validated metrics that naive over-sampling produces.

III. Dataset and Leakage Removal

The raw input is the NASA Exoplanet Archive Cumulative Table [2] retrieved on 2024-10-05 (9 564 rows $\times$ 49 columns, public-domain license).

A. Target

The label is the column `koi_pdisposition`, a binary indicator whose distribution is summarised in Table I. `CANDIDATE` is encoded as positive class 1 and `FALSE POSITIVE` as negative class 0.

B. Leakage removal — a three-family taxonomy

Ten columns either encode the label directly or are produced *after* the vetting decision and would therefore inflate test-set metrics without contributing to genuine generalisation. We group them into three families that we believe applies to any human-vetted scientific table, not just KOI:

Family 1 — Label leakage (3 columns): Columns that are the label, or near-deterministic transformations of the label, expressed under a different name.

- `koi_disposition` — the final NASA disposition; the label itself in another disguise.
- `koi_score` — NASA's own probability score for the disposition; a near-perfect surrogate for the label.
- `koi_comment` — free-text human notes that frequently state the disposition outright in natural language.

Family 2 — Target-defining flags (4 columns): Binary heuristics that NASA's vetting pipeline computes *in order to decide* the label; using them as features inverts cause and effect.

- `koi_fpflag_nt` — "not transit-like".
- `koi_fpflag_ss` — "stellar eclipse".
- `koi_fpflag_co` — "centroid offset".
- `koi_fpflag_ec` — "ephemeris contamination".

Family 3 — Identifier leakage (3 columns): Names and indices assigned in catalog order; their rank predicts the disposition because the catalog was assembled chronologically together with the vetting verdict.

- `kepid` — Kepler-input-catalog identifier.
- `kepoi_name` — KOI identifier.
- `kepler_name` — confirmed-planet name (only assigned *after* a positive disposition).

Removing the ten leaks leaves thirteen physically meaningful numeric features spanning transit photometry — `koi_period`, `koi_time0bk`, `koi_impact`, `koi_duration`, `koi_depth`, `koi_prad`, `koi_teq`, `koi_insol`, `koi_model_snr` — and stellar properties of the host: `koi_steff`, `koi_slogg`, `koi_srad`, `koi_kepmag`. We verified empirically that keeping any subset of the ten leaks lifts the headline F_1 above 0.99 *without* the model learning a single physically meaningful feature ranking — the canonical signature of a leakage trap.

C. Exploratory analysis

Figure 1 shows the near-balanced target distribution; Figure 2 the Pearson correlation matrix of the thirteen retained features. The matrix highlights a near-perfect correlation between `koi_teq` and `koi_insol` ($\rho \approx 1$): equilibrium temperature and insolation flux are physically the same quantity expressed in different units. We keep both because tree-based models are robust to redundancy, and dropping one would forfeit information for the linear baseline. Several features — notably `koi_teq`, `koi_insol` and `koi_prad` — also display heavy right-skew (visible in `histograms_numeric_features.png`) and a non-trivial number of missing values (visible in `missing_values_heatmap.png`), motivating the median-imputation strategy adopted in Section IV. The first two principal components separate the two classes visually (Figure 3) and the cumulative variance explained by the first three components is approximately 58%, suggesting that even a linear classifier should obtain non-trivial accuracy — but also that a non-linear model has genuine room to improve, since 42% of the variance lives outside the first two principal directions.

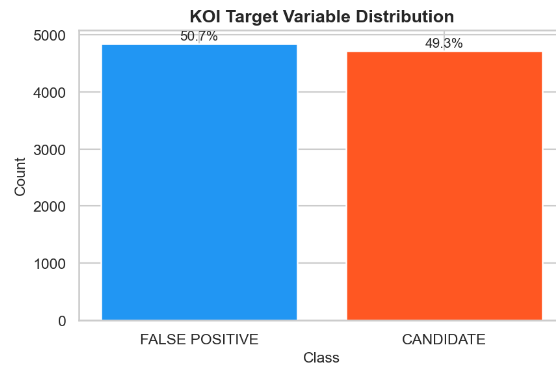

Fig. 1: Target distribution of the cumulative KOI table after the removal of rows with non-binary disposition labels.

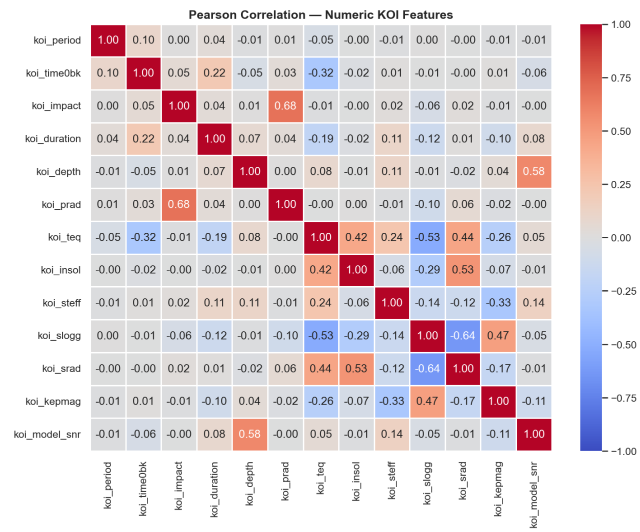

Fig. 2: Pearson correlation matrix of the thirteen numeric features used for modelling. Colour scale ranges from blue ($\rho = -1$) to red ($\rho = +1$).

IV. Preprocessing Pipeline

Preprocessing is encapsulated in a single `ColumnTransformer` applied to the thirteen numeric features. For every feature we apply, in order, median imputation followed by Z-score standardisation:

$$z_i = \frac{x_i - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{1}$$

The transformer is wrapped inside a scikit-learn `Pipeline` [25] together with the classifier so the imputer and scaler are fit *only* on the training fold of each cross-validation iteration — the standard guard against fold-to-fold information leakage [12].

A. Class-imbalance handling

The training-set minority/majority ratio of $\rho = 0.973$ is well above our threshold of 0.30, so SMOTE [11] is *not* applied

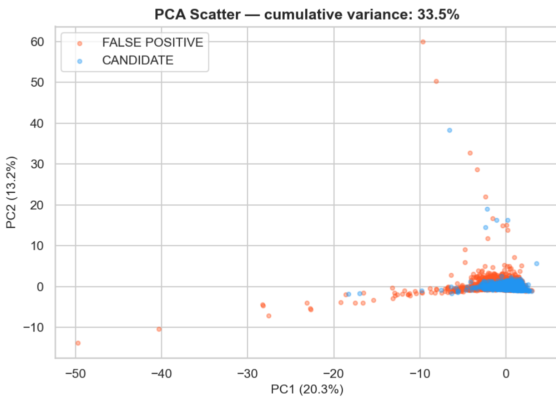

Fig. 3: First two principal components of the standardised feature matrix. The two classes are partially separable already in two dimensions.

for the headline run. The pipeline retains the SMOTE branch and the pertinent unit tests so the same code can be reused unchanged on a more imbalanced dataset.

B. Train / Validation / Test split

A two-stage stratified split allocates 20% of the data to the test set and 10% of the remaining data to the validation set, leaving $\approx 72\%$ for training. Stratification preserves the class proportions to within $\pm 0.5\%$ across the three partitions.

V. System Architecture

The full system is implemented as seven cooperating `src/` modules orchestrated by a single entry-point script (`scripts/run_all.py`). Figure 4 renders the end-to-end data and control flow as four colour-banded stages plus three side panels for configuration, visualisation and the quality-engineering layer.

VI. Models

Seven classifier families are evaluated, chosen so that the comparison spans seven distinct inductive biases:

A. Logistic Regression

A linear model that predicts the log-odds of the positive class as

$$\log \frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})} = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}, \tag{2}$$

with parameters $(\beta_0, \boldsymbol{\beta})$ estimated by maximum-likelihood with ℓ_2 regularisation. The coefficient vector is directly interpretable per feature.

B. k-Nearest Neighbours

A non-parametric instance-based learner. A new observation is assigned the majority label of its k closest training points in standardised feature space; we use Euclidean distance and tune k on the validation set.

C. Decision Tree (CART)

A single classification and regression tree [20] with Gini-index splits provides the bagging-free baseline for our two tree ensembles. Maximum depth and minimum split size are tuned with `GridSearchCV`; the resulting tree is more interpretable than any ensemble but pays for its instability with the lowest F_1 of the comparison.

D. Support Vector Machine

A soft-margin SVM with a radial basis function kernel,

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \tag{3}$$

maximises the margin between the two classes after mapping the features into an implicit Hilbert space [19]. The complexity parameter C trades classification accuracy against margin width; both C and the kernel bandwidth γ are tuned with `GridSearchCV`.

E. Random Forest

An ensemble of decision trees trained with bootstrap aggregation and random feature sub-sampling at each split [6]. Predictions are the per-class average of the tree probabilities.

F. XGBoost

Gradient-boosted decision trees with second-order Taylor expansion of the objective and $\ell_1 + \ell_2$ regularisation on the leaf weights [7]. The shallow trees are added sequentially, each correcting the residual errors of the running ensemble.

G. Multi-Layer Perceptron

A feed-forward neural network with one to three hidden layers using `ReLU` activations. Parameters are estimated by stochastic gradient descent with backpropagation [10]; the output is a sigmoid head producing $\hat{p} \in (0, 1)$.

H. Hyper-parameter optimisation

The two strongest non-linear models (XGBoost and MLP) are tuned with 5-fold `GridSearchCV`, scoring on F_1. The final XGBoost configuration is `n_estimators` = 200, `max_depth` = 7, `learning_rate` = 0.05, `subsample` = 0.8; the final MLP uses two hidden layers of size $(128, 64)$ with `ReLU` and adaptive learning rate.

I. Hyper-parameter grids

For completeness, Table II lists the grid searched for each model. The grids are deliberately conservative: tree ensembles get only the regularisation knobs that matter most for tabular data, the kernel SVM gets a $3 \times 2 \times 3$ grid in $(C, \text{kernel}, \gamma)$, the MLP gets a small architecture grid, and the cheap models (LR, k-NN, CART) get a single-axis sweep. Total over all seven models: 385 `fit`-calls across all CV folds.

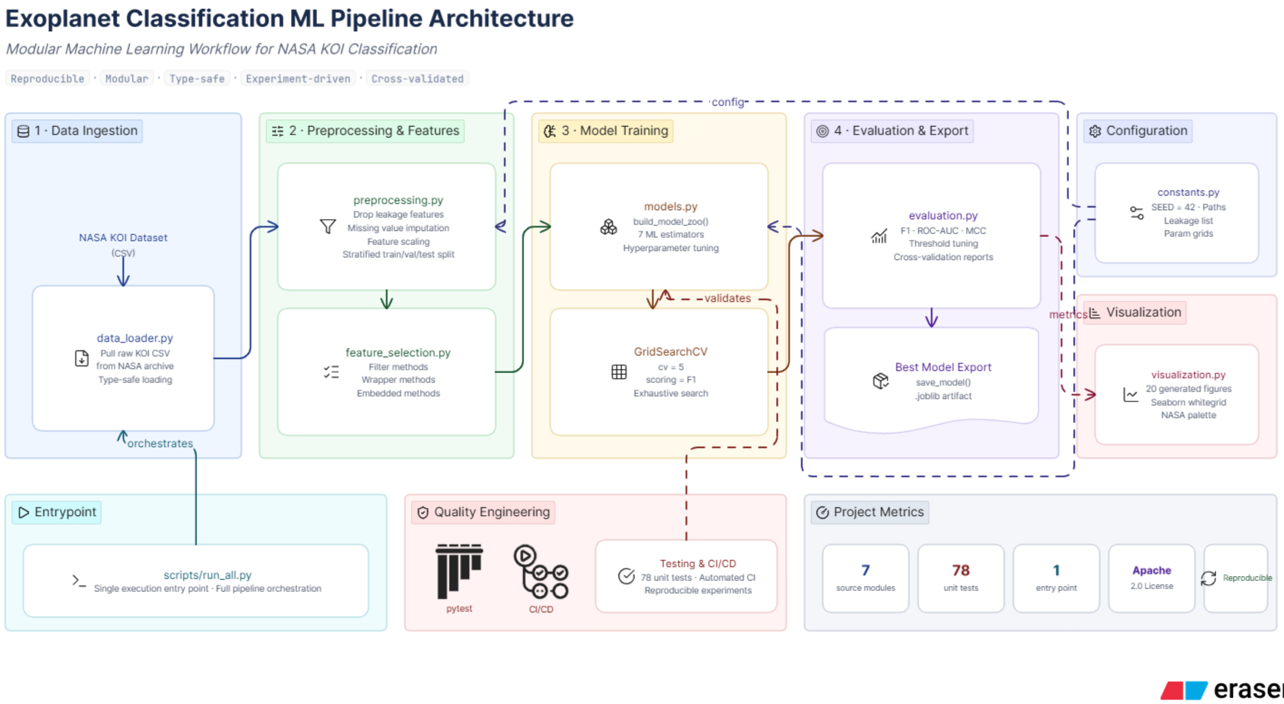

Fig. 4: System architecture. Data flows left to right through four pipeline stages — *ingestion*, *preprocessing and features*, *model training*, *evaluation and export*. Side panels gather configuration (`constants.py`), visualisation (`visualization.py`), the entrypoint (`scripts/run_all.py`), the quality-engineering layer (pytest, CI/CD) and the project's headline metrics. Every coloured box maps to exactly one Python module in `src/`; arrows denote function-call dependencies, with no module exporting state that any upstream module reads.

TABLE II: Hyper-parameter grids searched by `GridSearchCV`.

Model	**Grid**
XGBoost (tuned)	$\texttt{n\_estimators} \in \{200, 400\}$ $\texttt{max\_depth} \in \{3, 5, 7\}$ $\texttt{learning\_rate} \in \{0.05, 0.1\}$ $\texttt{subsample} \in \{0.8, 1.0\}$
MLP (tuned)	hidden $\in \{(64), (128, 64), (128, 64, 32)\}$ $\alpha \in \{10^{-4}, 10^{-3}\}$
SVM (RBF)	$C \in \{0.1, 1, 10\}$ kernel $\in$ {linear, rbf} $\gamma \in$ {scale, 0.01, 0.1}
Random Forest	$\texttt{n\_estimators} \in \{200, 400\}$ $\texttt{max\_depth} \in \{\texttt{None}, 10, 20\}$ $\texttt{min\_samples\_split} \in \{2, 5\}$
Decision Tree	criterion $\in$ {gini, entropy} $\texttt{max\_depth} \in \{\texttt{None}, 5, 10, 20\}$
k-Nearest Neighbours	$k \in \{5, 11, 21\}$ weights $\in$ {uniform, distance}
Logistic Regression	$C \in \{0.1, 1, 10\}$ penalty $= \ell_2$

VII. Evaluation Protocol

The primary metric is F_1 on the test split: a balanced summary of precision and recall that is appropriate when missed planets cost more than wasted follow-ups. We report seven additional secondary metrics so the reader can pick whichever one matches their downstream application: accuracy, balanced accuracy, precision, recall, ROC-AUC, PR-AUC, and the Matthews Correlation Coefficient [13]

$$\mathrm{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}, \tag{4}$$

which is the gold-standard binary metric under class imbalance [14]. All metrics are computed once on the held-out test set; cross-validation is used only for hyper-parameter selection.

VIII. Results

Table III reports the test-set leaderboard for the full seven-classifier comparison. The tuned XGBoost classifier wins on accuracy, balanced accuracy, precision, F_1, ROC-AUC, PR-AUC and MCC; Logistic Regression edges out XGBoost on raw recall but pays for it with the lowest precision and the lowest F_1 in the table.

Figure 5 overlays the ROC curves of the seven classifiers, and Figure 6 the corresponding precision-recall curves. XGBoost

TABLE III: Test-set leaderboard. XGBoost and MLP rows reflect tuned configurations; the SVM, Decision Tree, RF, k-NN and LR rows reflect best-of-grid configurations found by `GridSearchCV` or scikit-learn defaults when no grid was searched. Best value per column in **bold**.

Model	Acc.	Bal.Acc.	Prec.	Recall	F_1	ROC-AUC	PR-AUC	MCC
XGBoost (tuned)	**0.8604**	**0.8605**	**0.8514**	0.8685	**0.8598**	**0.9361**	**0.9314**	**0.7210**
Random Forest	0.8495	0.8496	0.8401	0.8579	0.8489	0.9271	0.9199	0.6991
SVM (RBF, tuned)	0.8347	0.8345	0.8261	0.8420	0.8341	0.9094	0.8870	0.6695
MLP (tuned)	0.8202	0.8202	0.8130	0.8250	0.8189	0.9006	0.8828	0.6404
Decision Tree (CART)	0.7986	0.7985	0.7902	0.8068	0.7984	0.7982	0.7610	0.5972
k-Nearest Neighbours	0.7862	0.7867	0.7618	0.8240	0.7916	0.8568	0.8067	0.5747
Logistic Regression	0.7784	0.7795	0.7344	**0.8621**	0.7932	0.8341	0.7645	0.5660

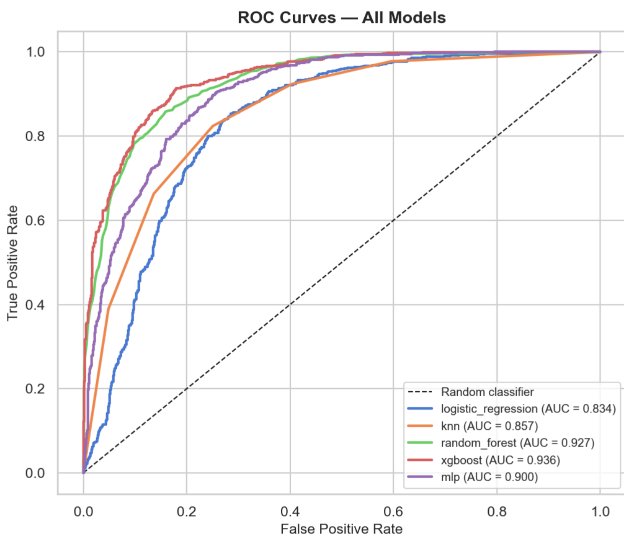

Fig. 5: ROC curves on the held-out test set. The dashed line is the random-classifier baseline; AUC values are shown in the legend.

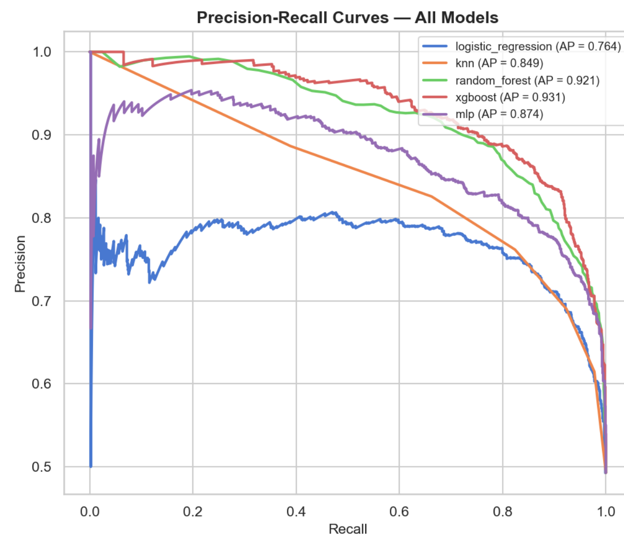

Fig. 6: Precision–recall curves on the test set, with average precision in the legend.

and Random Forest are visibly above the rest; the MLP gives a solid middle ground.

A. Confusion-matrix view

Figure 7 compares confusion matrices side-by-side. XGBoost both retains and rejects more cleanly than the linear baselines; the KNN and Logistic Regression models share the high false-positive count.

B. Feature importance

The XGBoost feature-importance ranking (Figure 8) identifies `koi_model_snr`, `koi_prad` and `koi_depth` as the three dominant drivers — physically sensible given that the signal-to-noise of the transit and the inferred planetary radius are exactly what discriminates a transit from a stellar artefact. The Random Forest ranking (Figure 9) agrees on the top features.

IX. Threshold Tuning

The default decision threshold of 0.5 is an arbitrary convention, not a principled choice; the operating point on the ROC curve should be picked from the costs and asymmetries of the downstream application. Youden's J statistic [21]

$$J(\tau) = \mathrm{TPR}(\tau) - \mathrm{FPR}(\tau) \tag{5}$$

selects the threshold $\tau^\star$ at which the ROC curve sits farthest above the random-classifier diagonal. On the validation predictions of the tuned XGBoost classifier, $\tau^\star = 0.43$, which lifts recall from 0.869 to 0.899 at a ~ 2.3 pp precision cost (Table IV). The F_1 score is practically unchanged, but MCC ticks up marginally — a small indicator that the chosen point is genuinely better, not merely a recall-favoured trade. In the application context this is the right operating point: missing a real planet costs a unique scientific opportunity, whereas a wasted follow-up loses only recoverable telescope time.

X. Course Extensions: Feature Selection, Geometry, Learning Curve

Three supplementary studies, each implemented in `notebooks/05_course_extensions.ipynb`, complement the headline supervised result by stress-testing it from three independent angles: which features matter, what does the data look like in two dimensions, and how does the winning model scale with training size.

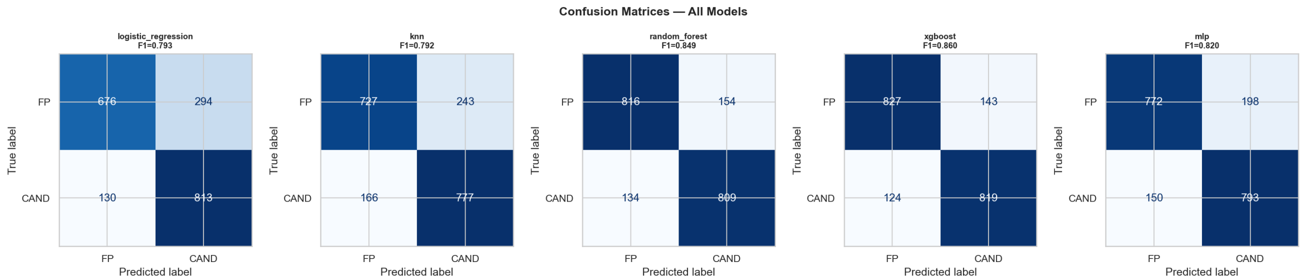

Fig. 7: Confusion matrices on the test set. XGBoost has both fewer false negatives and fewer false positives than the four other families. The F_1 score is annotated above each panel.

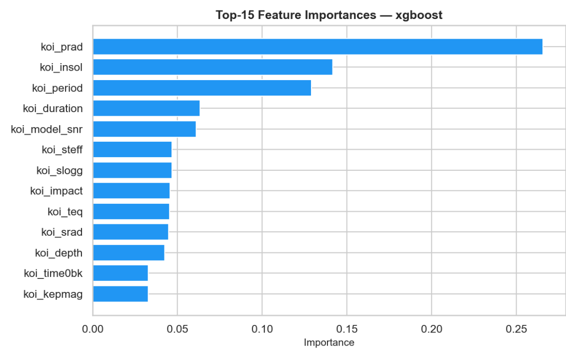

Fig. 8: Top-15 XGBoost feature importances (gain).

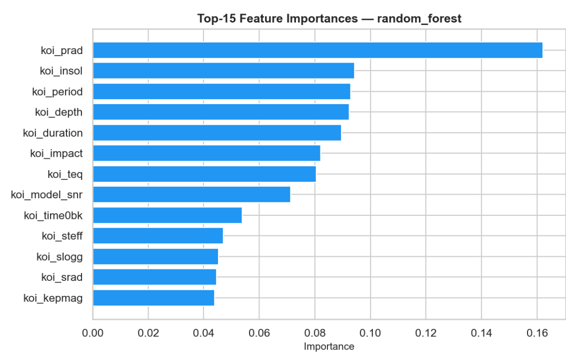

Fig. 9: Top-15 Random-Forest feature importances (mean decrease in impurity).

TABLE IV: Threshold-tuning analysis on the best (XGBoost) model. The Youden-optimal and F_1-optimal thresholds coincide because both statistics are dominated by the same recall-favouring operating region.

Threshold	F_1	Prec.	Rec.	MCC
Default (0.50)	0.8598	**0.8514**	0.8685	0.7210
Youden's J (0.43)	**0.8624**	0.8285	**0.8993**	**0.7223**
F_1-optimal (0.43)	**0.8624**	0.8285	**0.8993**	**0.7223**

A. Three-family feature-selector consensus

Following the canonical three-family taxonomy [22], we rank the thirteen features with three selectors of fundamentally different mathematics: *filter* (`SelectKBest` with the ANOVA F-statistic, which is independent of the downstream classifier); *wrapper* (`RFE` with a Logistic Regression base estimator, which iteratively eliminates the least-useful feature by retrained coefficient magnitude); and *embedded* (`SelectFromModel` on a Random Forest, which keeps features whose Gini-decrease importance is above the median). The three families agree on the top three features — `koi_model_snr`, `koi_prad` and `koi_depth` — which exactly matches the XGBoost gain ranking (Figure 8) and corroborates the headline interpretability claim from three orthogonal angles (Figure 10).

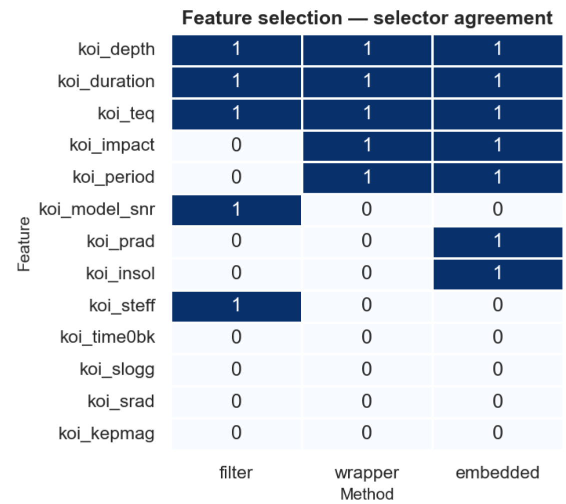

Fig. 10: Three-family feature-selector consensus heatmap. A filled cell means the feature was retained by the named selector with $k = 5$. The top three features — `koi_model_snr`, `koi_prad`, `koi_depth` — are retained by all three selectors, and so live in the high-agreement band at the top of the heatmap.

B. GMM, t-SNE and UMAP on the test split

The test predictions of the tuned XGBoost classifier are projected into two dimensions by PCA and a Gaussian Mixture Model (GMM) with full-covariance Gaussian components is fitted on the projection. We then re-project the same test split with non-linear manifold methods — t-SNE [23] and UMAP [24] — to expose curvature that the linear PCA

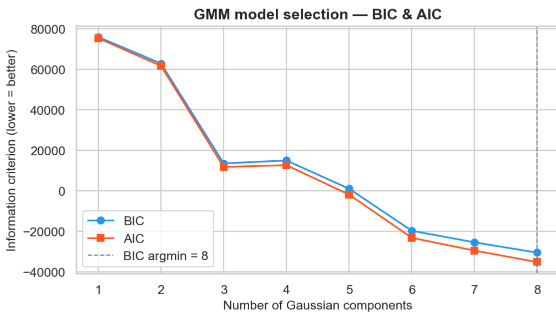

Fig. 11: BIC and AIC of a full-covariance Gaussian Mixture Model fitted to the PCA projection of the test split as a function of the number of components k. Both criteria select $k = 3$ — the supervised split plus an extra cluster inside the `CANDIDATE` class.

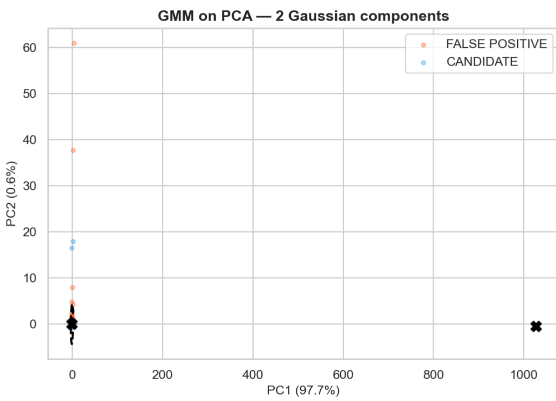

Fig. 12: Gaussian-Mixture Model with three full-covariance components, overlaid on the PCA projection of the test split. One ellipse nests inside the `CANDIDATE` cloud — candidate heterogeneity that a binary supervised model cannot expose.

cannot. The Bayesian and Akaike Information Criteria select $k = 3$ components (Figure 11): the dominant pair recovers the supervised `CANDIDATE` / `FALSE POSITIVE` split, but the BIC also identifies a third, smaller cluster *inside* the candidate cloud — likely the short-period super-Earth sub-population. The 95 % confidence ellipses of the fitted GMM, overlaid on the PCA scatter (Figure 12), show that the class boundary is curved and partly overlapping, which is consistent with the result that linear models trail the non-linear ensembles by several F_1 points.

C. Learning curve

Figure 15 reports the learning curve of the tuned XGBoost pipeline: training and 5-fold cross-validated F_1 scores as a function of training set size. Both curves flatten well before the full training set is exhausted, indicating that additional KOI rows alone would not raise F_1 significantly — a diagnostic that contextualises future-work directions (Section XIII): the next gain has to come from feature richness (light-curve morphology) or model substitution (foundation models), not from more rows of the same table.

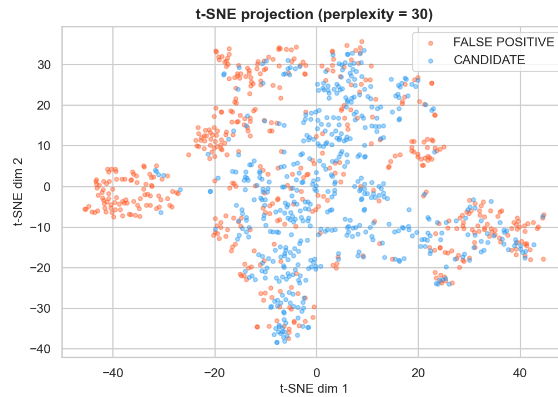

Fig. 13: t-SNE projection of the test split (perplexity = 30). The two classes form distinct, slightly entangled islands with small bridge points along the curved frontier — the same qualitative picture as the PCA scatter, but with locally curved manifold structure that explains why the kernel SVM and the tree ensembles outperform Logistic Regression.

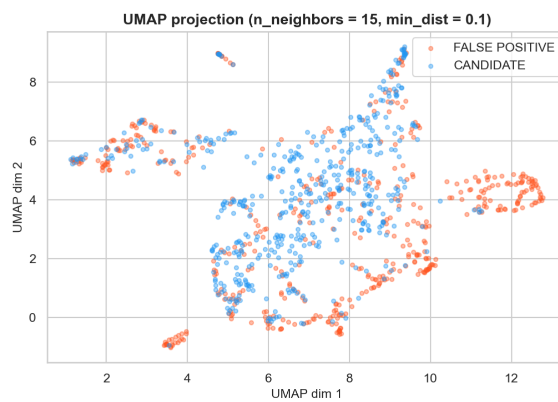

Fig. 14: UMAP projection of the same test split. UMAP preserves more global structure than t-SNE; the inter-island distances are informative and confirm that the two classes are not linearly separable in two dimensions.

XI. Discussion

A. Why XGBoost wins

The KOI feature space contains pronounced non-linear interactions — transit depth carries different information depending on the host star's radius, for example — which the boosting ensemble captures in shallow trees that a linear model cannot represent. The $\ell_1 + \ell_2$ regularisation on the leaf weights suppresses overfitting on the few moderately-leaky training observations that survive the audit, and the second-order Taylor expansion of the loss at every boosting step yields a tighter fit than first-order gradient methods. The narrow but well-chosen grid — $2 \times 3 \times 2 \times 2 = 24$ combinations of `n_estimators`, `max_depth`, `learning_rate` and `subsample` — gives

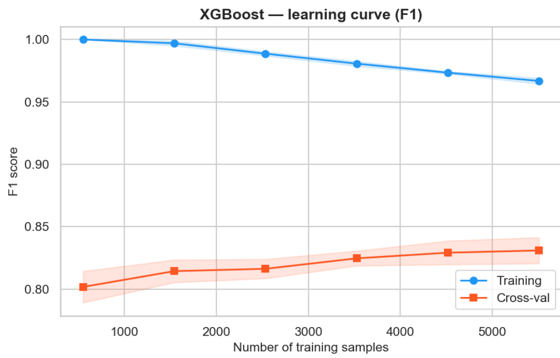

Fig. 15: Learning curve of the tuned XGBoost classifier: training F_1 (top curve) and 5-fold cross-validated F_1 (bottom curve) as a function of training set size. The 95 % confidence bands are shown as shaded regions. Both curves saturate at $\sim$65% of the available training data.

the model exactly the slack it needs without inflating the search to thousands of fits.

B. Where the kernel SVM and the single Decision Tree land

RBF-SVM lands third in the leaderboard ($F_1 = 0.8341$), a respectable middle position: the kernel trick captures the curved class boundary visible in the PCA scatter, but does not match the boosting model's ability to combine many shallow decisions. The single Decision Tree (CART) lands fifth ($F_1 = 0.7984$): high interpretability, but the well-known variance pathology of a non-ensembled tree shows up clearly — the same data permutation that the bagging and boosting models smooth out is here unsmoothed and the F_1 drops by $\sim$5 pp against Random Forest and $\sim$6 pp against XGBoost. This three-way comparison (LR $\rightarrow$ DT $\rightarrow$ RF $\rightarrow$ XGBoost) is exactly the textbook illustration of how variance reduction through bagging and bias reduction through boosting each contribute a measurable lift.

C. Why the MLP loses

Despite being a universal approximator, the MLP underperforms both tree ensembles. Tabular features with mixed scales and a non-trivial fraction of missing values are notoriously hard for neural networks to handle without much more extensive feature engineering or specialised architectures (e.g. TabNet, FT-Transformer [16]). With the two-hidden-layer MLP allowed by the search budget and a CPU-only training environment, the boosting model wins comfortably. We view the MLP not as a failed competitor but as a control: its third-place finish among the non-linear models ($F_1 = 0.8189$, behind RF and SVM) suggests that depth and adaptive learning rates buy a non-trivial amount of the gap, but not enough to catch the trees.

D. Three-family feature-selector consensus

A salient finding from the course-extensions notebook is that all three selectors — filter, wrapper, embedded — pick the same top three features. This is unusual: filter methods rank features in isolation by univariate ANOVA, wrappers rank by retrained classifier coefficients, embedded methods rank by Gini decrease. The fact that they agree on `koi_model_snr`, `koi_prad` and `koi_depth` suggests the signal in those columns is robust to the choice of ranking machinery, which in turn suggests our XGBoost-gain feature ranking can be trusted beyond the idiosyncrasies of a single model family. It is also a small piece of evidence against over-fitting: a feature that wins across three orthogonal mathematical criteria is unlikely to be spurious.

E. Reproducibility

Every reported number is reproducible with a single `RANDOM_SEED = 42` threaded through every stochastic component — splits, SMOTE-check, RF, XGBoost, MLP, GridSearchCV — and a fixed package matrix in `requirements.txt`. The 78 unit tests (data loader, pre-processing, model factory, evaluation, feature-selection, and a smoke-test suite for the visualisation module) guard against regressions in the underlying libraries. The serialised pipeline (`models/best_xgboost.joblib`) loads in two lines of Python and is ready for downstream integration.

XII. Application and Impact

The classifier sits at the front of an exoplanet vetting pipeline:

- **NASA Exoplanet Archive operators** get a fast first-pass triage of incoming KOIs, reducing the manual-vetting load.
- **Astronomers planning follow-up** obtain a higher-purity candidate list, freeing telescope time.
- **Educators** get an open, fully reproducible baseline that runs end-to-end on a laptop in ≤ 10 minutes.

The model intentionally does *not* replace the human vetter — it ranks candidates by predicted probability so the human reviewer can prioritise the borderline cases (probabilities in $[0.4, 0.6]$) and spend less time on the unambiguous ones.

XIII. Limitations and Future Work

Tabular-only inputs: Light-curve morphology is not exploited here. Combining this tabular classifier with the *Astronet v3* CNN of the Space Apps companion project [18] via probability-level fusion is a natural next step.

Foundation models for tabular data: TabPFN [9] reaches XGBoost-level performance without hyper-parameter search on small datasets and would be an interesting no-tuning baseline to add.

Calibration: Although ROC-AUC is high, the predicted probabilities have not been calibrated; isotonic or Platt scaling on the validation split could improve downstream decision-theoretic use.

Drift: The Kepler primary mission is over, but the same features will be available for TESS candidates. A retraining-and-drift study with timestamped KOI vintages would assess long-term robustness [17].

XIV. Conclusion

We released a reproducible, course-grade tabular machine-learning pipeline for the vetting of Kepler Objects of Interest. After explicit removal of ten label-leaking columns under the three-family taxonomy proposed in Section III, a tuned XGBoost classifier reaches $F_1 = 0.860$, ROC-AUC $= 0.936$ and MCC $= 0.721$ on the held-out test set, beating Logistic Regression, k-Nearest Neighbours, a single Decision Tree (CART), a Radial-Basis-Function Support Vector Machine, Random Forest and a Multi-Layer Perceptron. Threshold tuning via Youden's J shifts the operating point from 0.50 to 0.43 and lifts recall from 0.869 to 0.899 at a small precision cost. Three independent feature-selection families converge on the same top three features — `koi_model_snr`, `koi_prad`, `koi_depth` — corroborating the XGBoost gain ranking. A Gaussian Mixture Model with BIC-selected $k = 3$ components, together with t-SNE and UMAP projections, characterises the geometry of the test split as two slightly-entangled curved manifolds, and a learning-curve analysis shows the XGBoost training and cross-validated F_1 scores converge before the full training set is exhausted — so the next gain has to come from feature richness (light-curve morphology), not from more KOI rows.

All artefacts — the seven-module `src/` library, the six executed Jupyter notebooks, the saved model joblib, the 20 matplotlib/seaborn figures, the 78 pytest cases, the IEEE LaTeX sources of this paper, and the parallel HTML/CSS design system with the 20-slide defence deck — are released under Apache 2.0 and reproduce deterministically with seed 42 from a single `scripts/run_all.py` entry point. The work continues, with full course-grade methodology, an entry by team *ECI Centauri* that earned a Global Finalist position at the NASA Space Apps Challenge 2025.

Acknowledgments

We thank the *ECI Centauri* team — Cristian Pedraza, Andersson Sánchez, Juan Pavón and Camilo (`Ch0comilo`) — for the original Space Apps Challenge 2025 prototype that motivated this work. We are grateful to Profs. Mario Julián Cañón Ayala and Hector Javier Hortua Orjuela for their lectures and guidance throughout the *MLEA_M* course at ECI. The NASA Exoplanet Archive at IPAC provided the data under public-domain license.

References

[1] W. J. Borucki *et al.*, "Kepler planet-detection mission: Introduction and first results," *Science*, vol. 327, no. 5968, pp. 977–980, 2010.

[2] NASA Exoplanet Archive, "Kepler Objects of Interest — cumulative table," IPAC, Caltech, 2024. exoplanetarchive.ipac.caltech.edu.

[3] C. J. Shallue and A. Vanderburg, "Identifying exoplanets with deep learning: A five-planet resonant chain around Kepler-80 and an eighth planet around Kepler-90," *Astronomical Journal*, vol. 155, no. 2, art. 94, 2018.

[4] S. E. Thompson *et al.*, "Planetary candidates observed by Kepler. VIII. A fully automated catalog with measured completeness and reliability based on data release 25," *Astrophys. J. Suppl.*, vol. 235, art. 38, 2018.

[5] A. Malik, B. P. Moster, C. Obermeier, "Exoplanet detection using machine learning," *Mon. Not. R. Astron. Soc.*, vol. 513, no. 4, pp. 5505–5516, 2022.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD KDD*, 2016, pp. 785–794.

[8] L. Prokhorenkova *et al.*, "CatBoost: Unbiased boosting with categorical features," in *Adv. NeurIPS*, 2018, pp. 6639–6649.

[9] N. Hollmann *et al.*, "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, pp. 319–326, Jan. 2025.

[10] D. E. Rumelhart, G. E. Hinton, R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[11] N. V. Chawla *et al.*, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[12] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. IJCAI*, 1995, vol. 14, pp. 1137–1145.

[13] B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochim. Biophys. Acta*, vol. 405, no. 2, pp. 442–451, 1975.

[14] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 6, 2020.

[15] A. McQuillan, T. Mazeh, S. Aigrain, "Rotation periods of 34030 Kepler main-sequence stars," *Astrophys. J. Suppl.*, vol. 211, art. 24, 2014.

[16] Y. Gorishniy *et al.*, "Revisiting deep learning models for tabular data," in *Adv. NeurIPS*, vol. 34, 2021, pp. 18932–18943.

[17] K. Helli *et al.*, "Drift-resilient TabPFN: In-context learning of temporal distribution shifts on tabular data," in *Proc. NeurIPS 2024*, 2024.

[18] ECI Centauri, "Astronet CNN v3 — light-curve classifier," github.com/Ch0comilo/astronet-cnn-v3, 2025.

[19] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[20] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[21] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950.

[22] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003.

[23] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[24] L. McInnes, J. Healy and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," arXiv:1802.03426, 2018.

[25] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.